Интенсив по Go (2 вечера) — План и инструкции для ИИ-агента

Часть 1. Системный промпт и контекст для ИИ-агента (AGENTS.md)

Скопируй этот блок в системный промпт нового чата или в файл `AGENTS.md` в корне учебного репозитория по Go.

Контекст студента

- **Бэкграунд:** Уверенно владеет Java Core (ООП, коллекции, Stream API, `record`, иммутабельный `Builder`, исключения, JUnit 5).
- **Базы данных и сеть:** Спроектировал схему в PostgreSQL (3NF, индексы, транзакции, миграции Flyway), реализовал слой доступа к данным на чистом JDBC (`PreparedStatement`, ручной маппинг `ResultSet`, пул соединений HikariCP). Написал с нуля многопоточный TCP-чат и собственный HTTP/1.1-сервер на сокетах (`ServerSocket`, `ExecutorService`, кастомный парсер HTTP, Jackson), а также перенёс REST API на Spring Boot и Spring Data JPA.
- **Что НЕ нужно объяснять с нуля:** Базовые понятия клиент-серверной архитектуры, протоколы TCP/IP и HTTP (методы, заголовки, статус-коды), формат JSON, концепцию REST, основы реляционных БД, SQL и зачем нужны интерфейсы/слоистая архитектура.
- **Цель спринта:** За 2 вечера (по 3–4 часа) освоить ядро языка Go, его философию, модель памяти, конкурентность и стандартный пакет `net/http` на контрасте с Java, переписав мини-версию уже знакомого домена (`Task Planner`) на идиоматичный Go.

Правила работы для ИИ-агента (Роль Ментора и Ревьюера)

1. **Код пишет студент, а не агент:** Не выдавай готовое решение целиком. Давай короткие синтаксические справки (5–15 строк) для демонстрации идиомы Go, после чего формулируй чёткое пошаговое техническое задание (ТЗ), которое студент должен написать сам.
2. **Обучение через контраст с Java:** Объясняй каждую новую конструкцию Go через сравнение с тем, как студент уже делал это в Java (например: `struct` против `class`/`record`, явный возврат `error` против `try-catch`,

`defer` против `try-with-resources`, неявные интерфейсы против `implements`, горутины против потоков `ExecutorService`).

3. **Жёсткое идиоматическое код-ревью:** Главный враг Java-разработчика в Go — попытка писать «Java-код синтаксисом Go». На этапе ревью безжалостно указывай на неидиоматичные решения:
    - Геттеры с префиксом `Get` (в Go пишут `task.Title()` или обращаются к полю напрямую, а не `task.GetTitle()`).
    - Огромные интерфейсы на 10 методов в пакете реализации вместо маленьких интерфейсов на 1–3 метода в пакете потребителя (consumer-side interfaces).
    - Игнорирование ошибок через `_` или потерю контекста при возврате ошибки без `fmt.Errorf("...: %w", err)`.
    - Передачу тяжёлых структур по значению там, где нужна мутация, или неоправданное использование указателей там, где структура маленькая и иммутабельная.
4. **Цикл работы над каждым шагом:**
    - Шаг А: Краткая теория (контраст Java vs Go) + проверочный вопрос на понимание «под капотом».
    - Шаг Б: Чёткое практическое задание для самостоятельного написания кода.
    - Шаг В: Ревью кода студента, разбор ошибок, рефакторинг до идиоматичного состояния.

Часть 2. Учебный план: Модуль Go Crash Course (2 вечера)

Цель: Понять ментальную модель Go (простота, композиция, явная обработка ошибок, CSP-конкурентность) и собрать рабочий многопоточный REST API для Task Planner на стандартной библиотеке Go без сторонних фреймворков.

Сравнительная таблица: Смена ментальной модели (Java → Go)

Концепция	Как привык в Java	Как принято в Go
Сборка и зависимости	Maven (`pom.xml`), долгая сборка, JVM байт-код	`go mod init`, `go.mod`, мгновенная компиляция в нативный бинарник

Концепция	Как привык в Java	Как принято в Go
Инкапсуляция	Модификаторы `public`, `private`, `protected`, `package-private`	Регистр первой буквы: `Task` (публичный / exported), `task` (приватный для пакета)
Модели и ООП	Классы, наследование (`extends`), `record`, `Task.Builder`	Структуры (`struct`), композиция (встраивание структур), фабричные функции `NewTask(...)`
Интерфейсы	Явная декларация `class SQLTaskRepo implements TaskRepo`	Неявная реализация (утиная типизация): если методы совпадают — интерфейс реализован
Память и передача данных	Все объекты — ссылки в куче (кроме примитивов)	Всё передаётся по значению (копируется); явные указатели `*T` и взятие адреса `&x`
Динамические массивы	`ArrayList<T>` (объект-обёртка над массивом)	`slice` (`[]T`) — структура из 3 машинных слов: указатель на массив, `len`, `cap`
Обработка ошибок	Исключения (`throw`, `try-catch-finally`), стек-трейсы	Ошибки как значения (`func() (Task, error)`), проверка `if err != nil`
Освобождение ресурсов	`try-with-resources` (`AutoCloseable`)	Ключевое слово `defer` (выполняется перед выходом из функции в порядке LIFO)
Конкурентность	Тяжёлые потоки ОС (`Thread`, `ExecutorService`), `synchronized`	Легковесные горутины (`go fn()`), каналы (`chan`), `select`, `sync.Mutex`, `context.Context`

Концепция	Как привык в Java	Как принято в Go
HTTP и JSON	Spring Web (`@RestController`) или `ServerSocket` + Jackson	Стандартные пакеты `net/http` (`http.ServeMux`) и `encoding/json` (теги `` `json:"title"` ``)

Вечер 1 (3–4 часа): Ядро языка, память, структуры, интерфейсы и ошибки

Блок 1.1. Инструментарий, пакеты, указатели и структуры (1 час)

- **Теория:**
  - Инициализация модуля (`go mod init tasker`), точка входа `package main` и `func main()`.
  - Экспортируемые и неэкспортируемые идентификаторы (заглавная против строчной буквы).
  - Указатели (`*T`, `&x`): почему в Go всё передаётся строго по значению и когда функция получает копию структуры, а когда — адрес оригинала.
  - Структуры (`struct`), методы со значением-получателем (`func (t Task) ...`) против методов с указателем-получателем (`func (t *Task) ...`).
  - Отсутствие конструкторов и перегрузки методов: идиома фабричной функции `NewTask(...)` и паттерн Functional Options (вместо привычного `Task.Builder`).
- **Практика 1:**
  - Инициализировать проект и создать пакет `internal/domain`.
  - Описать типы `Status` и `Priority` (через типизированные константы `type Status string` или `iota`).
  - Создать структуру `Task` (поля: `ID`, `Title`, `Description`, `Status`, `Priority`, `Deadline` типа `time.Time`).
  - Написать функцию-конструктор `NewTask(...) (*Task, error)`, которая валидирует пустой `Title` и выставляет дефолтные `Status` и `Priority`.
  - Добавить метод `(t *Task) MarkDone()`, меняющий статус задачи.

Блок 1.2. Слайсы, мапы, неявные интерфейсы и ошибки (1.5–2 часа)

- **Теория:**

    - Внутреннее устройство `slice` (`ptr`, `len`, `cap`): почему при передаче слайса в функцию изменение элемента по индексу видно снаружи, а результат `append` — нет (и как работает аллокация при превышении `cap`).
    - Хеш-таблица `map[K]V`: чтение с проверкой существования ключа (`val, ok := m[id]`), порядок итерации (всегда случайный намеренно).
    - Обработка ошибок как значений: интерфейс `error`, создание сигнальных ошибок (`var ErrTaskNotFound = errors.New("task not found")`), оборачивание (`fmt.Errorf("...: %w", err)`) и проверка через `errors.Is` / `errors.As`.
    - Инструкция `defer` и как она заменяет `finally` и `try-with-resources`.
    - Неявные интерфейсы: почему в Go говорят *«Accept interfaces, return structs»* и объявляют интерфейс там, где он используется.
- **Практика 2:**
    - Создать пакет `internal/storage` и реализовать `MemoryStore` на базе `map[int64]domain.Task` и счётчика ID.
    - Реализовать методы `Save(t domain.Task) domain.Task`, `GetByID(id int64) (domain.Task, error)`, `ListAll() []domain.Task`, `Delete(id int64) error`.
    - При отсутствии задачи возвращать типизированную ошибку `ErrTaskNotFound`.
- **Практика 3 (Тестирование):**
    - Написать свой первый Table-Driven Test (табличный тест — главный стандарт тестирования в Go) в файле `storage_test.go` с использованием стандартного пакета `testing`.

Вечер 2 (3–4 часа): Конкурентность, Context и стандартный HTTP-сервер

Блок 2.1. Горутины, примитивы пакета `sync`, каналы и `context` (1.5 часа)

- **Теория:**
    - Горутины против потоков ОС: почему можно запустить 100 000 горутин (стек от 2 КБ, планировщик GMP в рантайме Go).
    - Проблема гонки данных (Data Race) в Go: почему стандартная `map` падает с фатальной ошибкой `concurrent map read and map write` при одновременном доступе из разных горутин.

    - Защита состояния: `sync.Mutex` и `sync.RWMutex` (`Lock`/`Unlock`, `RLock`/`RUnlock` в связке с `defer`). Ожидание завершения через `sync.WaitGroup`.
    - Философия *«Do not communicate by sharing memory; instead, share memory by communicating»*: небуферизированные и буферизированные каналы (`chan`), оператор `select`.
    - Пакет `context.Context`: дерево контекстов, отмена операций по таймауту (`context.WithTimeout`) и сигналу (`context.WithCancel`).
- **Практика 4:**
    - Сделать `MemoryStore` потокобезопасным с помощью `sync.RWMutex` (чтение через `RLock`, запись через `Lock`).
    - Написать конкурентный тест, в котором 50 горутин одновременно пишут и читают задачи через `sync.WaitGroup`, и запустить его с детектором гонок: `go test -race ./...`.
    - Написать фоновый воркер `StartOverdueChecker(ctx context.Context, store ...)` в отдельной горутине, который с помощью `time.NewTicker` и `select` периодически проверяет просроченные задачи и корректно останавливается при отмене `ctx.Done()`.

Блок 2.2. Нативный REST API на `net/http` и Graceful Shutdown (1.5–2 часа)

- **Теория:**
    - Как устроен `net/http` (на контрасте с твоим `SimpleHttpServer` на Java): интерфейс `http.Handler` (всего один метод `ServeHTTP(w http.ResponseWriter, r *http.Request)`) и функция-адаптер `http.HandlerFunc`.
    - Как `net/http` под капотом сам запускает отдельную горутину на каждое входящее соединение (больше не нужно вручную настраивать `ExecutorService`).
    - Современный роутинг в стандартном `http.ServeMux` (Go 1.22+): маршруты вида `"GET /tasks"`, `"POST /tasks"`, `"GET /tasks/{id}"`, извлечение параметра через `r.PathValue("id")`.
    - Пакет `encoding/json`: структурные теги (`` `json:"title"` ``), потоковое чтение `json.NewDecoder(r.Body).Decode(&dto)` и запись ответа.
    - Graceful Shutdown сервера через `signal.NotifyContext` и `server.Shutdown(ctx)`.
- **Практика 5:**

- Создать пакет `internal/httpapi` и реализовать обработчики для `GET /tasks`, `POST /tasks`, `GET /tasks/{id}`, `DELETE /tasks/{id}`.
- При получении `ErrTaskNotFound` из хранилища (проверка через `errors.Is`) возвращать статус `404 Not Found`, при невалидном JSON — `400 Bad Request`, при создании — `201 Created`.
- Собрать всё приложение в `cmd/tasker/main.go`: инициализировать `MemoryStore`, запустить фоновый `OverdueChecker`, поднять `http.Server` на порту `8080` и настроить Graceful Shutdown по `Ctrl+C` (`SIGINT`/`SIGTERM`).
- Прогнать знакомый набор команд `curl` для проверки всех сценариев.

Контрольные вопросы для самопроверки после 2 вечеров

1. Что произойдёт, если передать слайс в функцию и внутри неё вызвать `append`? Увидит ли вызывающая функция новые элементы и почему?
2. В чём разница между объявлением метода `func (t Task) Update()` и `func (t *Task) Update()`?
3. Почему в Go интерфейс принято объявлять в том пакете, который вызывает метод (например, в `service` или `httpapi`), а не в пакете `storage`?
4. Чем горутина отличается от потока ОС в Java (`java.lang.Thread`) и почему переключение между горутинами дешевле?
5. Зачем первым аргументом в методы репозиториев и сервисов в Go передают `ctx context.Context`?
6. Что произойдёт, если в Go записать значение в закрытый канал (`close(ch)`) или в `nil`-канал?

Источники для быстрого погружения

- **Интерактивный тур:** A Tour of Go — официальная песочница (проходится за 1.5–2 часа).
- **Справочник с примерами:** Go by Example — короткие выжимки кода по каждой конструкции языка.
- **Стиль и идиомы:** Effective Go — официальный гид от создателей языка по тому, как писать на Go, а не переносить паттерны из Java/C++.
- **Документация стандартной библиотеки:** pkg.go.dev/net/http и pkg.go.dev/context.